06_anlaysisIssueWeights

July 9, 2026

[1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from consts import *
import numpy as np
import scipy.stats as ss
from itertools import combinations
from statannotations.Annotator import Annotator
from scipy.stats import linregress

plt.rcParams.update({"font.size":9})
plt.rcParams.update({"figure.figsize":(16/2.54, 9/2.54)})
sns.set_style("ticks")
sns.set_context("paper")
```

0.0.1 Annotation Helper Functions

[2]:
```
def compute_pairwise_stats(data, x, y, hue, order, hue_order,
                           test="welch", correction="bonferroni",
                           correction_scope="per_x"):
    """
    Compute pairwise stats for every (x, hue1, hue2) combo.
    test: "welch" | "students" | "mannwhitney"
    correction_scope: "per_x" (correct within each x-category's pairs only)
                      or "global" (correct across every pair in the whole plot)
    Returns a DataFrame with one row per (x, g1, g2).
    """
    rows = []
    for xc in order:
        sub = data[data[x] == xc]
        for g1, g2 in combinations(hue_order, 2):
            v1 = sub.loc[sub[hue] == g1, y].dropna().to_numpy()
            v2 = sub.loc[sub[hue] == g2, y].dropna().to_numpy()
            if len(v1) < 2 or len(v2) < 2:
                stat, p = np.nan, np.nan
            elif test == "welch":
                stat, p = ss.ttest_ind(v1, v2, equal_var=False)
            elif test == "students":
```

```
                stat, p = ss.ttest_ind(v1, v2, equal_var=True)
            elif test == "mannwhitney":
                stat, p = ss.mannwhitneyu(v1, v2, alternative="two-sided")
            else:
                raise ValueError(test)
            rows.append(dict(x=xc, g1=g1, g2=g2, n1=len(v1), n2=len(v2),
                             mean1=v1.mean() if len(v1) else np.nan,
                             mean2=v2.mean() if len(v2) else np.nan,
                             stat=stat, p_raw=p))
    res = pd.DataFrame(rows)

    if correction == "bonferroni":
        if correction_scope == "per_x":
            res["n_tests"] = res.groupby("x")["p_raw"].transform("count")
        else:
            res["n_tests"] = res["p_raw"].notna().sum()
        res["p_adj"] = (res["p_raw"] * res["n_tests"]).clip(upper=1.0)
    else:
        res["p_adj"] = res["p_raw"]

    return res


def stars_for(p, thresholds=((1e-3, "***"), (1e-2, "**"), (0.05, "*"), (1.0,␣
 ↪"ns"))):
    if pd.isna(p):
        return ""
    for cutoff, sym in thresholds:
        if p <= cutoff:
            return sym
    return "ns"


def hue_dodge_offset(hue_order, width=0.8):
    """Replicates seaborn's default dodge offsets for n hue levels."""
    n = len(hue_order)
    sub_w = width / n
    offsets = {h: -width/2 + sub_w*(i + 0.5) for i, h in enumerate(hue_order)}
    return offsets


def draw_sig_brackets(ax, stats_df, order, hue_order,
                      hide_ns=True, width=0.8,
                      y_pad=0.02, bracket_gap=0.035,
                      tick_h=0.008, fontsize=8, lw=0.8):
    """
    Draws brackets above the data for each x-category using positions
```

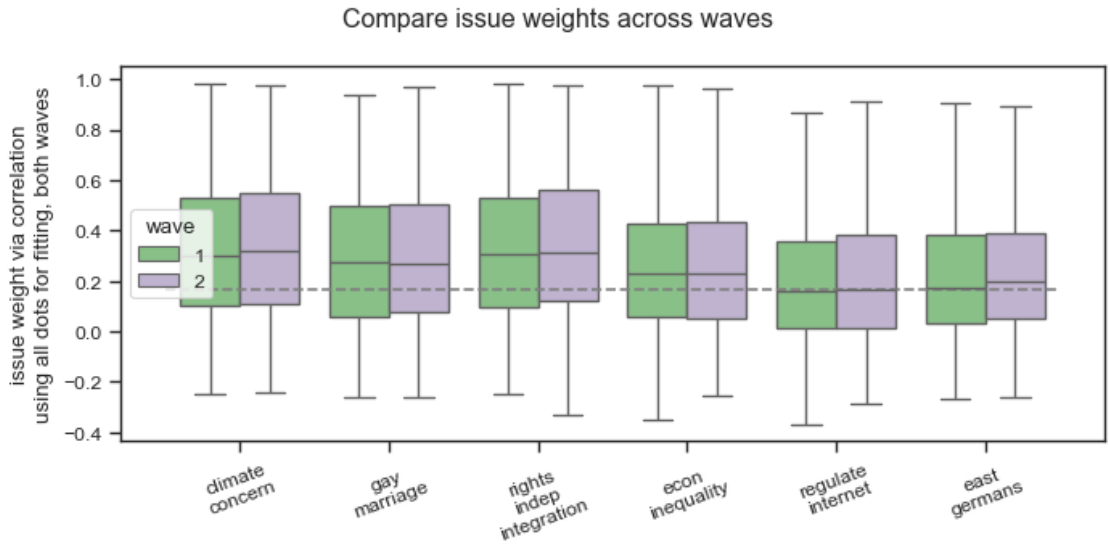
```
    computed with the same dodge math seaborn uses.
    stats_df must have columns: x, g1, g2, p_adj (from compute_pairwise_stats).
    """
    offsets = hue_dodge_offset(hue_order, width=width)
    x_index = {xc: i for i, xc in enumerate(order)}

    for xc in order:
        rows = stats_df[stats_df["x"] == xc].copy()
        if hide_ns:
            rows = rows[rows["p_adj"] <= 0.05]
        if rows.empty:
            continue

        # stack narrower spans lower, wider spans higher, so brackets don't␣
↪cross
        rows["span"] = rows.apply(lambda r: abs(offsets[r.g1] - offsets[r.g2]),␣
↪axis=1)
        rows = rows.sort_values("span")

        # base y = current top of the axes data area for this x-category
        base_y = ax.get_ylim()[1] - y_pad  # start just under the top; adjust␣
↪if needed
        # better: start just above the tallest bar/whisker actually plotted.
        # We estimate using ax's current children is fragile, so instead
        # give an explicit start_y as a parameter (see usage below).

        for i, (_, r) in enumerate(rows.iterrows()):
            xi = x_index[xc]
            x1 = xi + offsets[r.g1]
            x2 = xi + offsets[r.g2]
            y = base_y - i * bracket_gap  # stack downward from top, or invert␣
↪as needed

            ax.plot([x1, x1, x2, x2],
                    [y - tick_h, y, y, y - tick_h],
                    lw=lw, color="k", clip_on=False)
            ax.text((x1 + x2) / 2, y, stars_for(r.p_adj),
                    ha="center", va="bottom", fontsize=fontsize, clip_on=False)
```

[3]:
```
LRcuts = [0,0.33,0.67,1.]
fitmode = "fitAllDots"
```

[4]:
```
df_p = pd.read_csv("processed_data/
↪2026-07-07_data_processed_participant_withAllIssueWeights.csv")
df_diff = pd.read_csv("processed_data/
↪2026-07-07_data_processed_differences_withAllIssueWeights.csv")
```

1 Compare issue weights across the two waves

[5]:
```
k="corrP"
kname = "correlation"

strip = False
waves = [1,2]
alpha_cols = [f"{k}_alpha_{fitmode}_{q}" for q in questions_sc]
fig, ax = plt.subplots(1,1, figsize=(18/2.54, 9/2.54))
sns.boxplot(df_p[alpha_cols+["wave"]].melt(id_vars="wave", ).reset_index().
 ↪replace(dict(zip(alpha_cols, [q.replace("_", "\n") for q in␣
 ↪questions_sc]))), x="variable", y="value", hue="wave", palette=cmapWave,␣
 ↪hue_order=[1,2], fliersize=0, fill=not strip, legend=True)
if strip:
    sns.stripplot(df_p[alpha_cols+["wave"]].melt(id_vars="wave", ).
 ↪reset_index().replace(dict(zip(alpha_cols, [q.replace("_", "\n") for q in␣
 ↪questions_sc]))), x="variable", y="value", hue="wave", palette=cmapWave,␣
 ↪hue_order=[1,2], size=1, alpha=0.4, dodge=True, legend=False)
    sns.stripplot(df_p[alpha_cols+["wave"]].melt(id_vars="wave", ).
 ↪reset_index().replace(dict(zip(alpha_cols, [q.replace("_", "\n") for q in␣
 ↪questions_sc]))).groupby(["variable", "wave"])["value"].mean().
 ↪reset_index(), x="variable", y="value", hue="wave", palette=cmapWave,␣
 ↪hue_order=[1,2], size=10, alpha=0.8, dodge=True, marker="X", legend=False)
# sns.barplot(df_p[alpha_cols+["wave"]].melt(id_vars="wave", ).reset_index().
 ↪replace(dict(zip(alpha_cols, [q.replace("_", "\n") for q in␣
 ↪questions_sc]))), x="variable", y="value", hue="wave", palette=cmapWave,␣
 ↪hue_order=[1,2], estimator='mean')
fig.autofmt_xdate(rotation=20, ha="center")
ax.set_xlabel("")
ax.hlines(1/6, ax.get_xlim()[0], ax.get_xlim()[1], linestyles="--",␣
 ↪colors="grey")
ax.set_ylabel(f"issue weight via {kname}\nusing {'all dots for fitting' if␣
 ↪fitmode=='fitAllDots' else 'using only voter dots and self for fitting'},␣
 ↪{f'wave {waves[0]}' if len(waves)==1 else 'both waves'}")
fig.suptitle("Compare issue weights across waves")
fig.tight_layout()
```



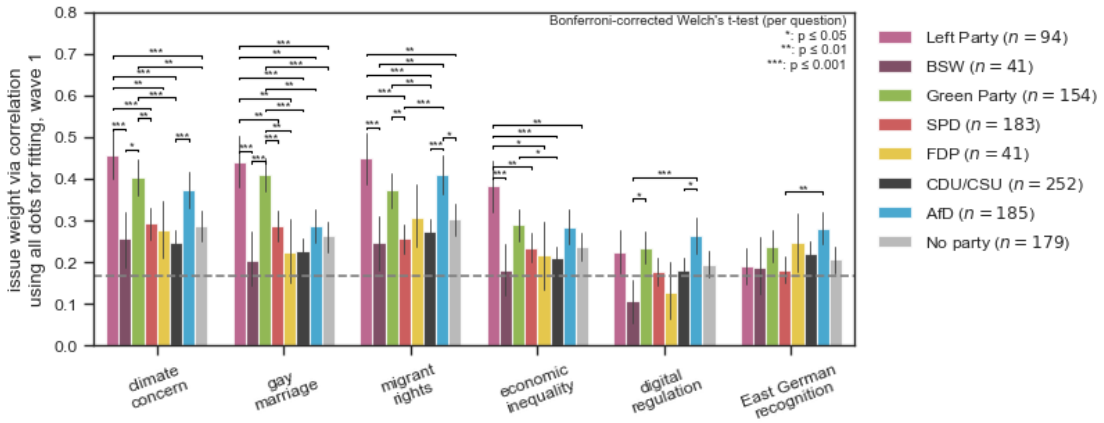
2 Compare issue weights by affiliated party

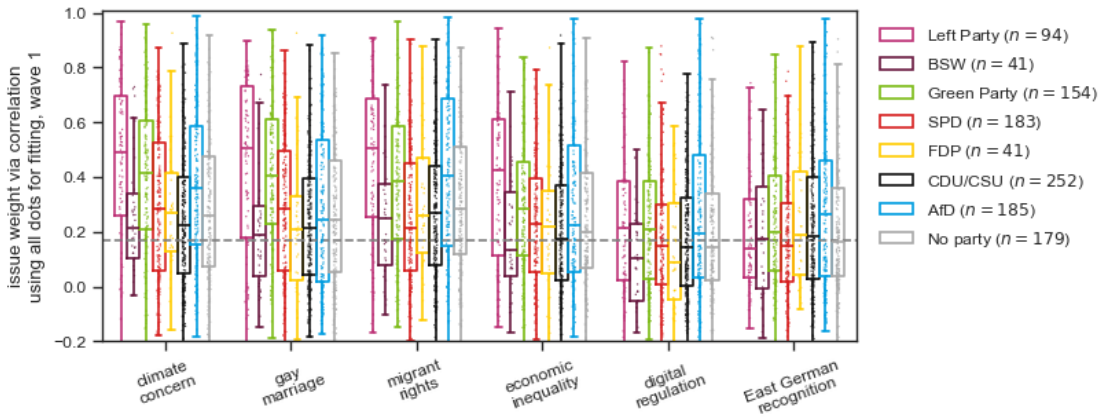
[6]:
```
# ----------------------------------------
# -------- By Party --------------
# ----------------------------------------
annotate = True
for strip in [False, True]:
    alpha_cols = [f"{k}_alpha_{fitmode}_{q}" for q in questions_sc]
    fig, ax = plt.subplots(1,1, figsize=(18/2.54, 9/2.54))
    ppp = parties + ["No party"]
    waves = [1]
    aa = df_p.loc[df_p.wave.isin(waves), alpha_cols+["party_close"]].
 ↪melt(id_vars="party_close", ).reset_index().replace(dict(zip(alpha_cols,␣
 ↪questions_sc)))
    aa["value"] = aa["value"]
    if strip:
        sns.stripplot(aa, x="variable", y="value", hue="party_close",␣
 ↪hue_order=ppp, palette=party_cmap, alpha=0.8, size=1, dodge=True,␣
 ↪legend=False)
        sns.boxplot(aa, x="variable", y="value", hue="party_close",␣
 ↪hue_order=ppp, palette=party_cmap, fliersize=0, fill = not strip)
    else:
        sns.barplot(aa, x="variable", y="value", hue="party_close",␣
 ↪hue_order=ppp, palette=party_cmap, err_kws={'linewidth': 0.6}, alpha=0.8,␣
 ↪estimator='mean', errorbar=('ci', 95), fill = not strip)
    ax.set_xticklabels([labelMap_nl[l.get_text()] for l in ax.
 ↪get_xticklabels()])
```

```
    fig.autofmt_xdate(rotation=20, ha="center")
    ax.set_xlabel("")
    ax.hlines(1/6, -0.5, len(questions_sc)-0.5, linestyles="--", colors="grey")
    ax.set_ylabel(f"issue weight via {kname}\nusing {'all dots for fitting' if␣
↪fitmode=='fitAllDots' else 'using only voter dots and self for fitting'},␣
↪{f'wave {waves[0]}' if len(waves)==1 else 'both waves'}")
    handles, labels = ax.get_legend_handles_labels()
    c = df_p.loc[df_p.wave.isin(waves), ["party_close"]].value_counts()
    labels = [f'{l} ($n={c[l]}$)' for l in labels]
    ax.legend(handles, labels, ncols=1, handlelength=2, columnspacing=0.5, ␣
↪frameon=False, bbox_to_anchor=(1.01,1))
    # --- significance annotations ---
    if annotate and not strip:
        stats_df = compute_pairwise_stats(
            aa, x="variable", y="value", hue="party_close",
            order=questions_sc, hue_order=list(ppp),
            test="welch",               # or "mannwhitney" if you go back to that
            correction="bonferroni",
            correction_scope="per_x",
        )

        # print(stats_df.sort_values("p_adj").head(20))

        grp_max = aa.groupby("variable")["value"].mean()
        offsets = hue_dodge_offset(list(ppp), width=0.8)
        x_index = {q: i for i, q in enumerate(questions_sc)}

        n_pairs = len(list(combinations(ppp, 2)))
        bracket_gap = 0.025
        tick_h = 0.005

        for q in questions_sc:
            rows = stats_df[(stats_df["x"] == q) & (stats_df["p_adj"] <= 0.05)].
↪copy()
            if rows.empty:
                continue
            rows["span"] = rows.apply(lambda r: abs(offsets[r.g1]-offsets[r.
↪g2]), axis=1)
            rows = rows.sort_values("span")  # narrow spans first, drawn lowest

            top = grp_max[q] + 0.15
            for i, (_, r) in enumerate(rows.iterrows()):
                xi = x_index[q]
                x1, x2 = xi + offsets[r.g1], xi + offsets[r.g2]
                y = top + i * bracket_gap
                ax.plot([x1, x1, x2, x2], [y-tick_h, y, y, y-tick_h],
                        lw=0.8, color="k", clip_on=False)
```

```
                ax.text((x1+x2)/2, y-2*tick_h, stars_for(r.p_adj),
                        ha="center", va="bottom", fontsize=7, clip_on=False)

        ax.text(0.99, 0.99,
            "Bonferroni-corrected Welch's t-test (per question)\n"
            "*: p  0.05\n**: p  0.01\n***: p  0.001",
            transform=ax.transAxes, fontsize=7,
            verticalalignment='top', horizontalalignment='right')

    if strip:
        ax.set_ylim(-0.2,0.32 if not "corr" in k else 1.01)
    else:
        ax.set_ylim(0,0.32 if not "corr" in k else 0.8)
    ax.set_xlim(-0.5,len(questions_sc)-0.5)
    plt.savefig(f"figs/issue_weights_{k}_{fitmode}_by_party{'_strip' if strip␣
↪else ''}.png", dpi=600)
```

```
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_59198/4100897836.py:1
7: UserWarning: set_ticklabels() should only be used with a fixed number of
ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels([labelMap_nl[l.get_text()] for l in ax.get_xticklabels()])
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_59198/4100897836.py:1
7: UserWarning: set_ticklabels() should only be used with a fixed number of
ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels([labelMap_nl[l.get_text()] for l in ax.get_xticklabels()])
```

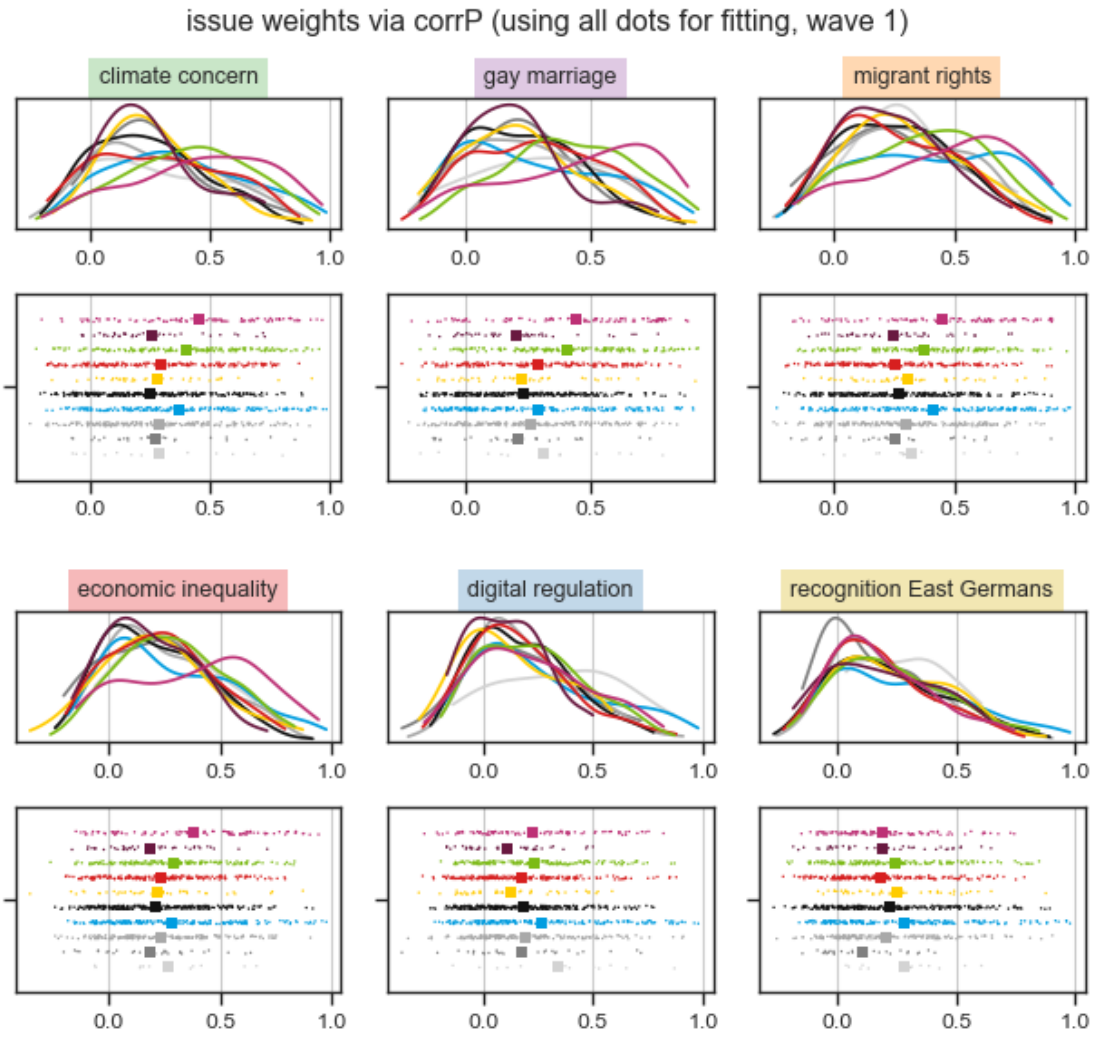




[7]:
```
# # --------- By Party Dist --------------
# alpha_cols = [f"{k}_alpha_{fitmode}_{q}" for q in questions_sc]
# fig, ax = plt.subplots(1,1, figsize=(18/2.54, 9/2.54))
# ppp = parties + ["No party"]
# waves = [1,2]
# aa = df_p.loc[df_p.wave.isin(waves), alpha_cols+["party_close"]].
 ↪melt(id_vars="party_close", ).reset_index().replace(dict(zip(alpha_cols,␣
 ↪questions_sc)), )
# aa["value"] = aa["value"]
# sns.boxplot(aa, x="variable", y="value", hue="party_close", hue_order=ppp,␣
 ↪palette=party_cmap, fliersize=0, showcaps=False,  medianprops={"linewidth":␣
 ↪3, "color":"k"}, notch=True, saturation=0.3)
# sns.stripplot(aa, x="variable", y="value", hue="party_close", hue_order=ppp,␣
 ↪palette=party_cmap, alpha=0.8, size=2, dodge=True, legend=False)
# fig.autofmt_xdate(rotation=20, ha="center")
# ax.set_xlabel("")
# ax.hlines(1/6, -0.5, len(questions_sc)-0.5, linestyles="--", colors="grey")
# ax.set_ylabel(f"issue weight via {kname}\nusing {'all dots for fitting' if␣
 ↪fitmode=='fitAllDots' else 'using only voter dots and self for fitting'},␣
 ↪{f'wave {waves[0]}' if len(waves)==1 else 'both waves'}")
# handles, labels = ax.get_legend_handles_labels()
# c = df_p.loc[df_p.wave.isin(waves), ["party_close"]].value_counts()
# labels = [f'{l}\n$n={c[l]}$' for l in labels]
# ax.legend(handles, labels, ncols=1, handlelength=2, columnspacing=0.5, ␣
 ↪frameon=False, bbox_to_anchor=(1.01,1))
# ax.set_ylim(0,0.32 if not "corr" in k else 0.9)
# ax.set_xlim(-0.5,len(questions_sc)-0.5)
```

[8]:
```
waves = [1]
# Party --> Issue weight distribution
```

```
fig, axs = plt.subplots(5,3, sharex=False, sharey=False, figsize=(16/2.54, 15/2.
↪54), gridspec_kw={"height_ratios":[0.7,1,0.01,0.7,1]})
for ax in axs[2,:]:
    ax.axis("off")
axq = {}
for ax, q in zip(axs[[1,4],:].flatten(), questions_sc):
    ax.grid("x")
    aa = df_p.loc[df_p.wave.isin(waves)]
    sns.stripplot(aa.groupby("party_close")[f"{k}_alpha_{fitmode}_{q}"].mean().
↪reset_index(),  x=f"{k}_alpha_{fitmode}_{q}", hue="party_close",␣
↪hue_order=parties_full, palette=party_cmap, ax=ax, legend=False, dodge=True,␣
↪size=4, marker="s")
    sns.stripplot(aa, x=f"{k}_alpha_{fitmode}_{q}", hue="party_close",␣
↪hue_order=parties_full, palette=party_cmap, ax=ax, legend=False, dodge=True,␣
↪size=1)
    ax.set_xlabel("")
    axq[q] = ax
for ax, q in zip(axs[[0,3],:].flatten(), questions_sc):
    ax.grid("x")
    # sns.boxplot(df_p, x=f"{k}_alpha_{q}", hue="party_close",␣
↪hue_order=parties_full, palette=party_cmap, ax=ax,
    #             fill=True, width=0.5, legend=False, fliersize=0)
    sns.kdeplot(aa, x=f"{k}_alpha_{fitmode}_{q}", hue="party_close",␣
↪hue_order=parties_full, palette=party_cmap, ax=ax,
                fill=False, cut=0, legend=False, common_norm=False)
    ax.set_title(labelMap[q], bbox=dict(facecolor=cmapQuestions[q], alpha=0.3,␣
↪edgecolor='none', pad=4), fontsize=9)
    ax.set_ylim(0,)
    ax.set_yticks([])
    ax.set_ylabel("")
    ax.set_xlabel("")
    ax.sharex(axq[q])
    #ax.set_xlim(0.0,0.5 if not "corr" in k else 1.0)
fig.suptitle(f"issue weights via {k} (using {'all dots for fitting' if␣
↪fitmode=='fitAllDots' else 'using only voter dots and self for fitting'},␣
↪{f'wave {waves[0]}' if len(waves)==1 else 'both waves'})")
fig.tight_layout()
```



3 Compare issue weights across LR

[9]:
```
# ----------------------------------------
# --------- By LR --------------
# ----------------------------------------
annotate=True
for strip in [False, True]:
    alpha_cols = [f"{k}_alpha_{fitmode}_{q}" for q in questions_sc]
    fig, ax = plt.subplots(1,1, figsize=(18/2.54, 9/2.54))
    waves = [1]
    df_p["lr_label"]= pd.cut(df_p.lr, bins=LRcuts, labels=cmapLR.keys())
    aa = df_p.loc[df_p.wave.isin(waves), alpha_cols+["lr_label"]].
 ↪melt(id_vars="lr_label", ).reset_index().replace(dict(zip(alpha_cols,␣
 ↪questions_sc)))
```

```
    aa["value"] = aa["value"]
    if strip:
        sns.boxplot(aa, x="variable", y="value", hue="lr_label",␣
↪hue_order=cmapLR.keys(), palette=cmapLR, fliersize=0, fill = not strip)
        sns.stripplot(aa, x="variable", y="value", hue="lr_label",␣
↪hue_order=cmapLR.keys(), palette=cmapLR, alpha=0.8, size=1, dodge=True,␣
↪legend=False, jitter=1/len(aa["lr_label"].unique()))
        sns.stripplot(aa.groupby(["lr_label", "variable"])["value"].mean().
↪reset_index(), x="variable", y="value", hue="lr_label", hue_order=cmapLR.
↪keys(), palette=cmapLR, alpha=0.8, size=10, marker="X", dodge=True,␣
↪legend=False, jitter=1/len(aa["lr_label"].unique()))
    else:
        sns.barplot(aa, x="variable", y="value", hue="lr_label",␣
↪hue_order=cmapLR.keys(), palette=cmapLR, err_kws={'linewidth': 0.6}, alpha=0.
↪8, estimator='mean', errorbar=('ci', 95), fill=False if strip else True)
    ax.set_xticklabels([labelMap_nl[l.get_text()] for l in ax.
↪get_xticklabels()])
    fig.autofmt_xdate(rotation=20, ha="center")
    ax.set_xlabel("")
    # ax.hlines(1/6, -0.5, len(questions_sc)-0.5, linestyles="--",␣
↪colors="grey")
    ax.hlines(0, -0.5, len(questions_sc)-0.5, linestyles="-", colors="k")
    ax.grid(axis="y")
    ax.set_ylabel(f"issue weight via {kname}\nusing {'all dots for fitting' if␣
↪fitmode=='fitAllDots' else 'using only voter dots and self for fitting'},␣
↪{f'wave {waves[0]}' if len(waves)==1 else 'both waves'}")
    # ax.set_ylabel(f"issue weight \n({k} kernel, {fitmode}, {f'wave␣
↪{waves[0]}' if len(waves)==1 else 'both waves'})")
    handles, labels = ax.get_legend_handles_labels()
    c = df_p.loc[df_p.wave.isin(waves), ["lr_label"]].value_counts()
    labels = [f'{l} ($n={c[l]}$)' for l in labels]
    ax.legend(handles, labels, ncols=3, handlelength=2, columnspacing=0.5, ␣
↪frameon=False)
    ax.set_ylim(-0.3,0.32 if not "corr" in k else 1)
    ax.set_xlim(-0.5,len(questions_sc)-0.5)
    # --- significance annotations ---
    if annotate and not strip:
        stats_df = compute_pairwise_stats(
            aa, x="variable", y="value", hue="lr_label",
            order=questions_sc, hue_order=list(cmapLR.keys()),
            test="welch",               # or "mannwhitney" if you go back to that
            correction="bonferroni",
            correction_scope="per_x",  # correct within each question's 3 pairs␣
↪only
        )
```

```
        # sanity check before trusting the plot -- print anything sub-alpha
        print(stats_df.sort_values("p_adj").head(20))

        # compute a real per-question ceiling from the data (mean's CI top, or␣
↪raw max)
        grp_max = aa.groupby("variable")["value"].mean()
        offsets = hue_dodge_offset(list(cmapLR.keys()), width=0.8)
        x_index = {q: i for i, q in enumerate(questions_sc)}

        n_pairs = len(list(combinations(cmapLR.keys(), 2)))
        bracket_gap = 0.07
        tick_h = 0.02

        for q in questions_sc:
            rows = stats_df[(stats_df["x"] == q) & (stats_df["p_adj"] <= 0.05)].
↪copy()
            if rows.empty:
                continue
            rows["span"] = rows.apply(lambda r: abs(offsets[r.g1]-offsets[r.
↪g2]), axis=1)
            rows = rows.sort_values("span")  # narrow spans first, drawn lowest

            top = grp_max[q] + 0.2
            for i, (_, r) in enumerate(rows.iterrows()):
                xi = x_index[q]
                x1, x2 = xi + offsets[r.g1], xi + offsets[r.g2]
                y = top + i * bracket_gap
                ax.plot([x1, x1, x2, x2], [y-tick_h, y, y, y-tick_h],
                        lw=0.8, color="k", clip_on=False)
                ax.text((x1+x2)/2, y-0.5*tick_h, stars_for(r.p_adj),
                        ha="center", va="bottom", fontsize=7, clip_on=False)

        ax.text(0.99, 0.99,
            "Bonferroni-corrected Welch's t-test (per question)\n"
            "*: p  0.05\n**: p  0.01\n***: p  0.001",
            transform=ax.transAxes, fontsize=7,
            verticalalignment='top', horizontalalignment='right')
    plt.savefig(f"figs/issue_weights_by_lr_{'strip' if strip else ''}.png",␣
↪dpi=600)
```

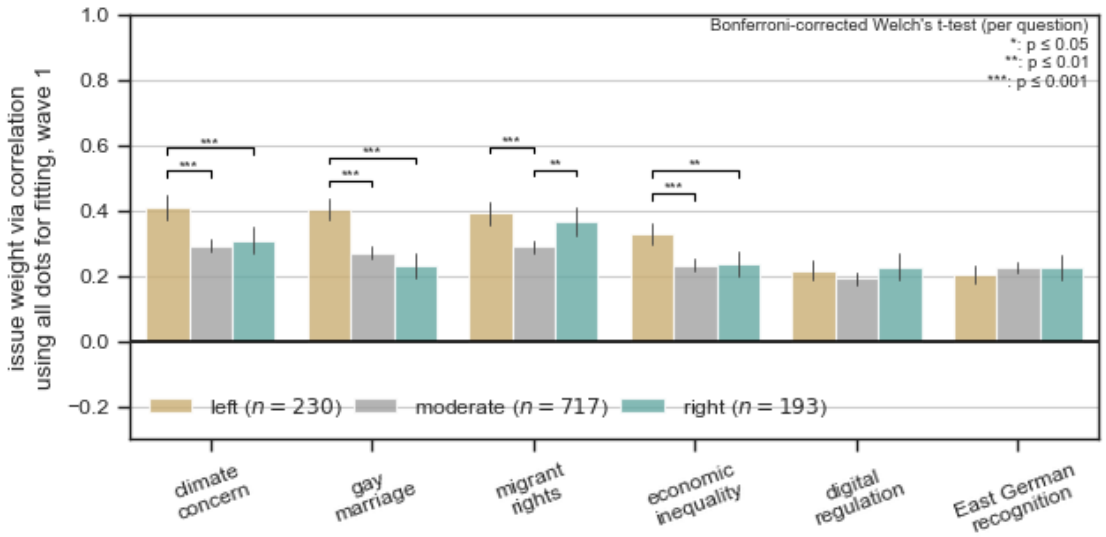
```
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_59198/1272815838.py:9
: PerformanceWarning: DataFrame is highly fragmented.  This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
  df_p["lr_label"]= pd.cut(df_p.lr, bins=LRcuts, labels=cmapLR.keys())
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_59198/1272815838.py:1
```

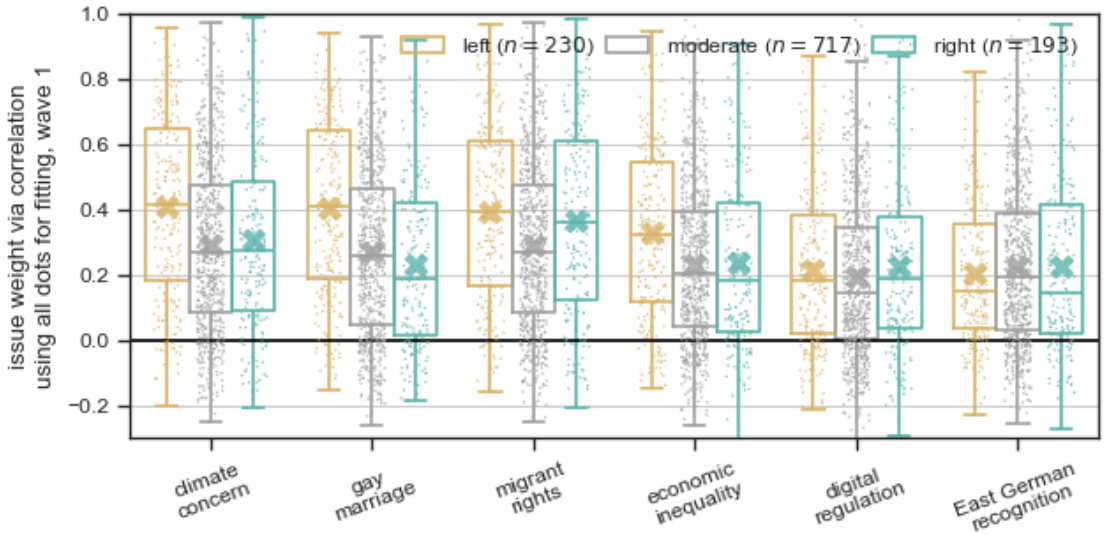
```
8: UserWarning: set_ticklabels() should only be used with a fixed number of
ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels([labelMap_nl[l.get_text()] for l in ax.get_xticklabels()])
```

	x	g1	g2	n1	n2	mean1	\
4	gay_marriage	left	right	230	193	0.403333	
3	gay_marriage	left	moderate	230	716	0.403333	
0	climate_concern	left	moderate	230	715	0.409164	
6	rights_indep_integration	left	moderate	230	715	0.392771	
9	econ_inequality	left	moderate	230	716	0.326177	
1	climate_concern	left	right	230	193	0.409164	
10	econ_inequality	left	right	230	193	0.326177	
8	rights_indep_integration	moderate	right	715	192	0.288739	
5	gay_marriage	moderate	right	716	193	0.270457	
14	regulate_internet	moderate	right	717	192	0.191848	
12	regulate_internet	left	moderate	230	717	0.216062	
15	east_germans	left	moderate	230	717	0.203894	
7	rights_indep_integration	left	right	230	192	0.392771	
16	east_germans	left	right	230	191	0.203894	
2	climate_concern	moderate	right	715	193	0.290638	
11	econ_inequality	moderate	right	716	193	0.232099	
13	regulate_internet	left	right	230	192	0.216062	
17	east_germans	moderate	right	717	191	0.223594	

	mean2	stat	p_raw	n_tests	p_adj
4	0.229244	6.754950	4.851332e-11	3	1.455399e-10
3	0.270457	6.530280	2.147559e-10	3	6.442676e-10
0	0.290638	5.713198	2.309575e-08	3	6.928726e-08
6	0.288739	5.110735	5.178165e-07	3	1.553450e-06
9	0.232099	4.881250	1.572817e-06	3	4.718452e-06
1	0.307480	3.717900	2.290322e-04	3	6.870966e-04
10	0.233947	3.618767	3.334685e-04	3	1.000406e-03
8	0.364319	-3.228334	1.398066e-03	3	4.194199e-03
5	0.229244	1.967602	5.001865e-02	3	1.500559e-01
14	0.225807	-1.570403	1.174397e-01	3	3.523192e-01
12	0.191848	1.322196	1.868610e-01	3	5.605829e-01
15	0.223594	-1.161533	2.460816e-01	3	7.382449e-01
7	0.364319	1.018209	3.092054e-01	3	9.276161e-01
16	0.224077	-0.828626	4.078621e-01	3	1.000000e+00
2	0.307480	-0.750606	4.535097e-01	3	1.000000e+00
11	0.233947	-0.087797	9.300993e-01	3	1.000000e+00
13	0.225807	-0.387863	6.983319e-01	3	1.000000e+00
17	0.224077	-0.022325	9.822052e-01	3	1.000000e+00

```
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_59198/1272815838.py:1
8: UserWarning: set_ticklabels() should only be used with a fixed number of
ticks, i.e. after set_ticks() or using a FixedLocator.
  ax.set_xticklabels([labelMap_nl[l.get_text()] for l in ax.get_xticklabels()])
```





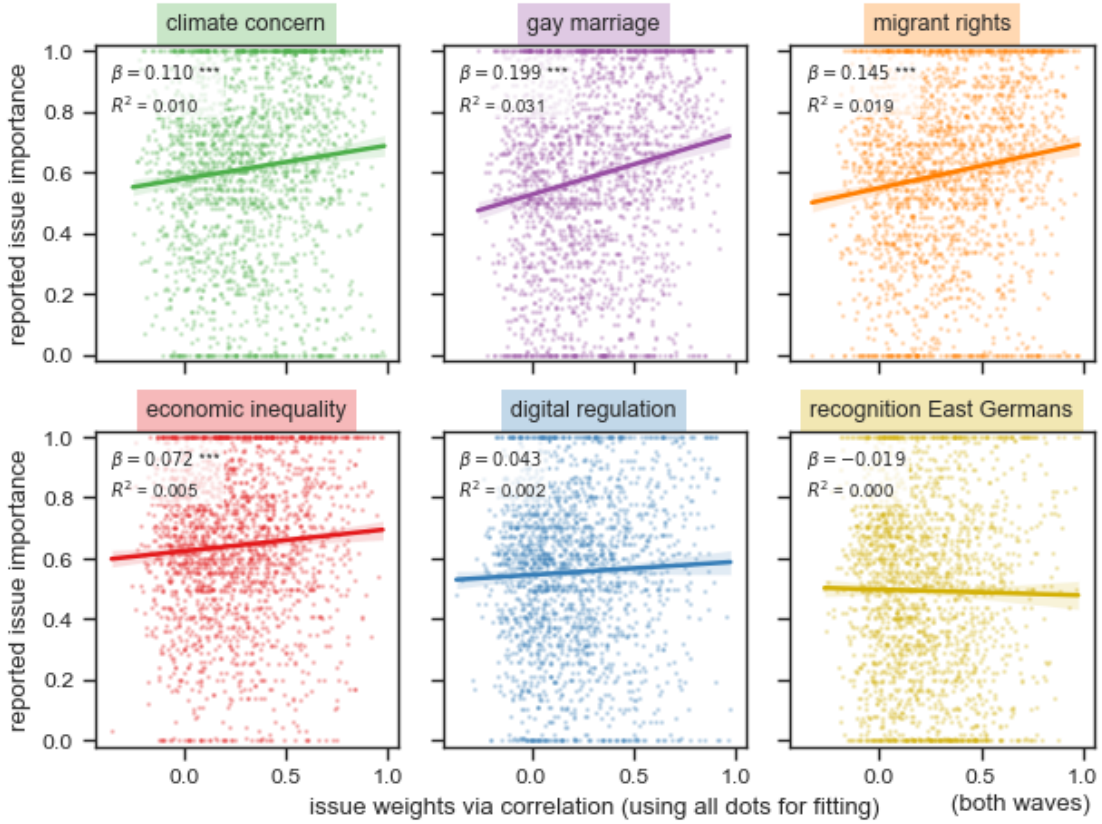
4 Other Analyses

4.0.1 Correlation Inferred vs Reported Issue Weights

[13]:
```
# ----------------------------------------
# --------- Correlation Inferred vs Reported Issue Weights -------------
# ----------------------------------------
#
waves = [1,2]
```

```
fig, axs = plt.subplots(2,3, sharex=True, sharey=True, figsize=(16/2.54, 12/2.
↪54))
for ax, q in zip(axs.flatten(), questions_sc):
    vary =  f"w_{q}"
    varx = f"{k}_alpha_{fitmode}_{q}"
    ax.grid("x")
    aa = df_p.loc[df_p.wave.isin(waves), [f"{k}_alpha_{fitmode}_{q}"]+[f"w_{q}"␣
↪for q in questions_sc]]
    aa[f"w_{q}_rel"] = aa[f"w_{q}"]/aa[[f"w_{q}" for q in questions_sc]].
↪sum(axis=1)
    aa = aa.dropna()

    sns.regplot(aa, x=varx, y =vary, color=cmapQuestions[q], ax=ax,␣
↪scatter_kws={"s":1, "alpha":0.2})
    ax.grid()
    ax.set_title(labelMap[q], bbox=dict(facecolor=cmapQuestions[q], alpha=0.3,␣
↪edgecolor='none', pad=4), fontsize=9)
    if not 'rel' in vary:
        ax.set_ylim(-0.02,1.02)
    ax.set_ylabel("")
    ax.set_xlabel("")
    #ax.set_xlim(0.0,0.5 if not "corr" in k else 1.0)
    print(q, f'{aa[[vary, varx]].corr().iloc[0,1]:.3f}',)
    res = linregress(aa[varx].values, aa[vary])
    p = res.pvalue
    ax.text(
        0.05, 0.95,
        fr"$\beta = {res.slope:.3f}$ "+(( "***" if p<0.001 else ("**" if p<0.01␣
↪else "*")) if p<0.05 else "")+f"\n$R^2$ = {res.rvalue**2:.3f}",
        transform=ax.transAxes,
        ha="left", va="top",
        fontsize=7,
        bbox=dict(facecolor="white", alpha=0.7, edgecolor="none", pad=1),
    )

axs[-1,-1].text(0.99, -0.22,f'(wave {waves[0]})' if len(waves)==1 else '(both␣
↪waves)', va="bottom", ha="right", transform=ax.transAxes)
axs[0,0].set_ylabel(f"{'relative ' if 'rel' in vary else ''}reported issue␣
↪importance")
axs[1,0].set_ylabel(f"{'relative ' if 'rel' in vary else ''}reported issue␣
↪importance")
axs[1,1].set_xlabel(f"issue weights via {kname} (using {'all dots for fitting'␣
↪if fitmode=='fitAllDots' else 'using only voter dots and self for␣
↪fitting'})")
fig.tight_layout()
plt.savefig("figs/correlation_w_corrP.png", dpi=600)
```

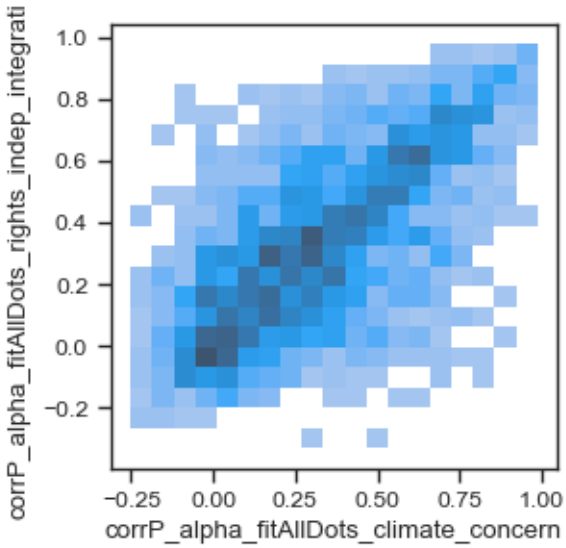
```
climate_concern 0.102
gay_marriage 0.175
rights_indep_integration 0.139
econ_inequality 0.074
regulate_internet 0.041
east_germans -0.016
```

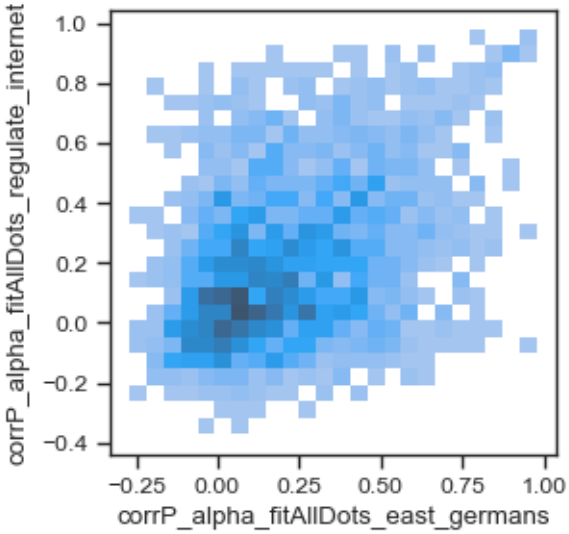


4.0.2 Correlation between issue weights

[11]:
```
# ----------------------------------------
# --------- Correlation between issue weights -------------
# ----------------------------------------
waves = [1,2]
fig, ax = plt.subplots(1,1,figsize=(3,3))
sns.histplot(df_p.loc[df_p.wave.isin(waves)], x=␣
 ↪f"{k}_alpha_{fitmode}_climate_concern", y =␣
 ↪f"{k}_alpha_{fitmode}_rights_indep_integration", ax=ax)
fig, ax = plt.subplots(1,1, figsize=(3,3))
sns.histplot(df_p.loc[df_p.wave.isin(waves)], x=␣
 ↪f"{k}_alpha_{fitmode}_east_germans", y =␣
 ↪f"{k}_alpha_{fitmode}_regulate_internet", ax=ax)
```

[11]:
```
<Axes: xlabel='corrP_alpha_fitAllDots_east_germans',
ylabel='corrP_alpha_fitAllDots_regulate_internet'>
```





4.0.3 Max issue weight by party

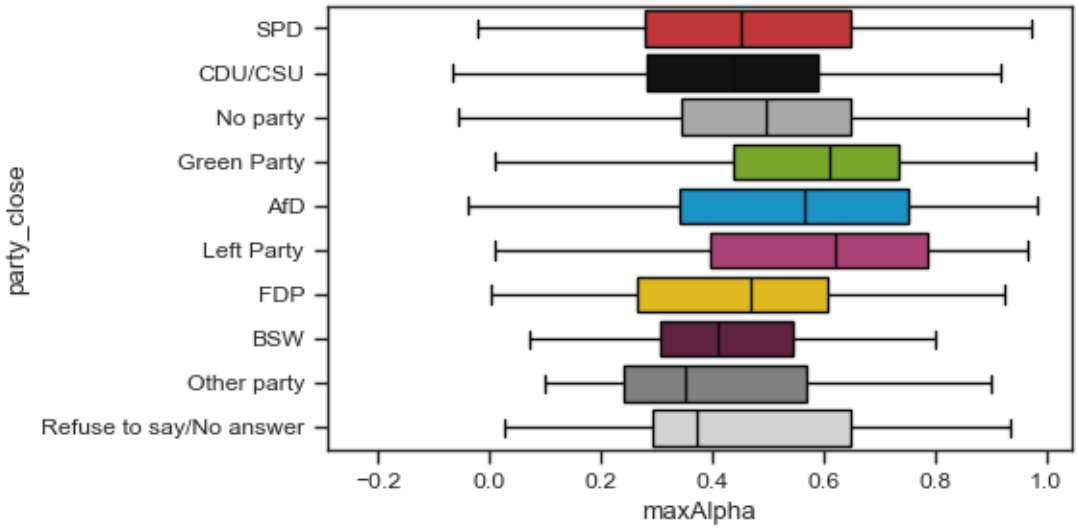
(only for correlation)

[12]:
```
# ---------------------------------------
# --------- MaxWeighta --------------
# ---------------------------------------
```

```
df_p["maxAlpha"] = df_p[alpha_cols].max(axis=1)
plt.figure(figsize=(5,3))
sns.boxplot(df_p.loc[df_p.wave.isin(waves)], y="party_close", x="maxAlpha",␣
 ↪hue="party_close", palette=party_cmap, fliersize=0, hue_order=parties_full)
#sns.histplot(df_p, x="maxAlpha", hue="party_close", palette=party_cmap,␣
 ↪kde=True, bins=np.linspace(0,1), stat="proportion", common_norm=False,␣
 ↪kde_kws={"cut":0})
```

```
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_59198/1577997786.py:4
: PerformanceWarning: DataFrame is highly fragmented.  This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
  df_p["maxAlpha"] = df_p[alpha_cols].max(axis=1)
```

[12]: <Axes: xlabel='maxAlpha', ylabel='party_close'>



[ ]: